Reagan Morgan

CS-320

August 17th, 2025

Project Two: Summary and Reflections Report

1. Summary

My approach was aligned with the software requirements by using the requirements as a basis for my testing. In all of my features, the class tests started with the requirements lined out for each variable (i.e. not null, certain character limit). Then, when moving to the Service classes, I implemented the requirement for the unique ID to not have any repeat numbers. I also made sure that each of the methods needed for the service classes outlined by the requirements did what was necessary from what was described. For example, in the Contact class, I needed to delete contacts when given the ID of a contact. From this, I developed two tests: one to test that the contact was deleted, and the other to test that the error would throw if someone entered a contact ID that did not exist within the ArrayList.

I know my tests for all 3 features and their respective classes (both the data class and the service class) were sound because of their code coverage. While I could have missed some specific edge cases or issues, I know that my code was up to standard because I tested for all the requirements that were laid out for the features. The coverage percentages for the data class and service class for each of the features respectively is listed below:

1. Contact – data class: 100% | service class: 97.6%

2. Task – data class: 100% | service class: 100%

3. Application – data class: 100% | service class: 100%

The contact service class had a lower amount of code coverage, but still within the industry standard of 80-100%, since I missed a branch of the logic I included in the for-loops for the ID validation (if it exists). I was unable to figure out why the same logic was fully covered in the latter two tests, but not within the contact feature even though I had similar, if not identical, tests for all 3 features when it came to ID validation.

To ensure that my code was technically sound, I made sure to refactor my code according to the feedback I received. A good example of this is seen in the first iteration of my ContactService class. I had many errors that neglected data encapsulation and the way some of the Assertion class methods worked. In the first two photos, you can see these errors when I directly access the contactService arrayList in the addContact test instead of using a getContact method and when I used multiple assertions in the deleteContact test when only the first one runs. To fix this, I added a getContact method and completely change the way I ran the deleteContact test by using the error handling logic I added in the class to make sure the contact was actually deleted by trying to retrieve it using getContact. You can see this in the last two screenshots.

```
@Test
void test_AddContact() {
    ContactService contactService = new ContactService();
    contactService.addContact("1", "John", "Doe", "1234567890", "123 Street St.");
    Assertions.assertEquals(1, contactService.contacts.size());
}
```

AddContact Test from the first iteration of ContactServiceTest

```
@Test
void test_DeleteContact() {
    ContactService contactService = new ContactService();
    contactService.addContact("1", "John", "Doe", "1234567890", "123 Street St.");
    contactService.addContact("2", "Jane", "Doe", "1234567890", "123 Street St.");
    Assertions.assertEquals("2", contactService.contacts.get(1).getID());
    contactService.deleteContact("2");
    Assertions.assertFalse(contactService.contacts.contains("2"));
}
```

DeleteContact Test from the first Iteration of ContactServiceTest

```java
@Test
void test_AddContact() {
    ContactService contactService = new ContactService();
    Contact contact = new Contact("1", "John", "Doe", "1234567890", "123 Street St.");
    contactService.addContact("1", "John", "Doe", "1234567890", "123 Street St.");
    Assertions.assertEquals(contact, contactService.getContact("1"));
}
```

AddContact Test from the second Iteration of ContactServiceTest

```java
@Test
void test_DeleteContact() {
    ContactService contactService = new ContactService();
    contactService.addContact("1", "John", "Doe", "1234567890", "123 Street St.");
    contactService.deleteContact("1");
    Assertions.assertThrows(IllegalArgumentException.class, () -> {
        contactService.getContact("1");
    });
}
```

DeleteContact Test from the second Iteration of ContactServiceTest

To ensure that my code was efficient, I tested each method of my code that contained logic to ensure proper function. I also made sure that my addContact, addTask, and addApplication methods were able to handle multiple elements as seen below.

```java
@Test void test_AddContact_MultipleElements() {
    ContactService contactService = new ContactService();
    Contact contact1 = new Contact("1", "John", "Doe", "1234567890", "123 Street St.");
    Contact contact2 = new Contact("2", "John", "Doe", "1234567890", "123 Street St.");
    Contact contact3 = new Contact("3", "John", "Doe", "1234567890", "123 Street St.");
    contactService.addContact("1", "John", "Doe", "1234567890", "123 Street St.");
    contactService.addContact("2", "John", "Doe", "1234567890", "123 Street St.");
    contactService.addContact("3", "John", "Doe", "1234567890", "123 Street St.");
    Assertions.assertAll(
            () ->  Assertions.assertEquals(contact1, contactService.getContact("1")),
            () ->  Assertions.assertEquals(contact2, contactService.getContact("2")),
            () ->  Assertions.assertEquals(contact3, contactService.getContact("3"))
            );
}
```

```java
@Test
void test_AddTask_MultipleElements_isSuccessful() {
    Task task1 = new Task("1", "Task Name", "this is a description.");
    Task task2 = new Task("2", "Task Name", "this is a description.");
    Task task3 = new Task("3", "Task Name", "this is a description.");
    TaskService taskService = new TaskService();
    taskService.addTask("1", "Task Name", "this is a description.");
    taskService.addTask("2", "Task Name", "this is a description.");
    taskService.addTask("3", "Task Name", "this is a description.");
    Assertions.assertAll("Grouped checking of task added.",
            () -> assertEquals(task1, taskService.getTask("1")),
            () -> assertEquals(task2, taskService.getTask("2")),
            () -> assertEquals(task3, taskService.getTask("3"))
    );

}
```

```java
@Test void test_AddContact_MultipleElements() {
    ContactService contactService = new ContactService();
    Contact contact1 = new Contact("1", "John", "Doe", "1234567890", "123 Street St.");
    Contact contact2 = new Contact("2", "John", "Doe", "1234567890", "123 Street St.");
    Contact contact3 = new Contact("3", "John", "Doe", "1234567890", "123 Street St.");
    contactService.addContact("1", "John", "Doe", "1234567890", "123 Street St.");
    contactService.addContact("2", "John", "Doe", "1234567890", "123 Street St.");
    contactService.addContact("3", "John", "Doe", "1234567890", "123 Street St.");
    Assertions.assertAll(
            () -> Assertions.assertEquals(contact1, contactService.getContact("1")),
            () -> Assertions.assertEquals(contact2, contactService.getContact("2")),
            () -> Assertions.assertEquals(contact3, contactService.getContact("3"))
            );
}
```

2. Reflection

The software testing technique I employed was Unit testing, specifically JUnit for Java. This type of software testing technique tests each method of the system in order to determine if it is working as intended. For my JUnit tests, I specifically targeted all the methods that had logic in them. This is because testing a method like a setter is inefficient as there is no logic to be tested, unlike a getter class that contains validation code as seen in all 3 of my features.

While I used Unit testing, there are other types of software testing that are important to learn. Some of these include:

- Black Box Testing: This type of testing relies on the tester having zero knowledge of the backend system and logic. An example of this using our previous

assignments would be a tester trying to create different contacts using the console or an actual GUI. The tester would input both valid and invalid types of data. In the case of the contacts, the tester would likely try to add a contact with correct ID, First Name, Last Name, Phone, and address. Then they would try it again with each one of the inputs being invalid. They could also try deleting contacts that are nonexistent or getting contacts that haven't been made yet.

- Integration Testing: This type of testing tests the interaction between two different modules. For example, if someone inputs data that is then passed to an authentication module, and then to an account module, this type of testing would check if the data was correctly transferred from the authentication module to the account module.
- System Testing: This type of testing encompasses everything within the system. This type of testing is usually used to make sure functional and non-functional requirements are met and are cohesive within the system. One way this may be integrated in testing would be having a tester act as an actual user, going through a login, exploration of pages, and finally a logout.

While acting as a software tester, I employed caution in multiple ways. One of these was making sure I refactored my code to make sure that all of my data was encapsulated. You can see this in the previous section where I go over how my code was technically sound. If data is not encapsulated, it can cause serious problems further down the road like bad actors corrupting saved data within the classes.

I tried to limit my bias by making sure that for every positive test case I had checking to make sure it functioned properly, I had multiple negative test cases to check for what would happen if user input was not in line with the specifications laid out in the guidelines for each feature. You

can see this for my deleteApplication method test in the screenshot below. I can definitely see why bias would be a concern if one was to develop and test their code at the same time, especially in a professional environment, because of the pressure of wanting to succeed in a timely manner. It can also be a concern if the software developer cannot figure out how to properly test their code in a sufficient manner, leading to only having simple tests that don't cover most or all edge cases.

```
@Test
void test_deleteApp_success() {
    ApplicationService service = new ApplicationService();
    service.addApplication("1", date, "Desc.");
    service.deleteApplication("1");
    Assertions.assertThrows(IllegalArgumentException.class, () -> {
        service.getApplication("1");
    });
}

@Test
void test_deleteAppInvalidID_fails() {
    ApplicationService service = new ApplicationService();
    service.addApplication("1", date, "Desc.");
    Assertions.assertThrows(IllegalArgumentException.class, () -> {
        service.deleteApplication("2");
    });
}
```

Being disciplined when it comes to software quality as a SWE professional is extremely important because it could not only impact your future career but also impact users if one is not committed to quality in their products. We can see in some of the discussion posts from this module that there are numerous times where Software Engineers either become complacent in their work or are pushed to be complacent due to the seemingly impossible deadlines laid out by the nontechnical people within a company. In the future, I plan to avoid technical debt as a practitioner by making sure every piece of code I write is meaningful and is above the standard for what I know. Mistakes will happen, but learning from them and making sure my code never has the same mistake again is what will allow me to constantly improve and achieve an above average standard. I will also make sure I communicate and push for extending deadlines if that is

within the realm of possibility as waiting longer to release quality code is better than having to constantly rework and update subpar code.